

MBA EM **ENGENHARIA
DE SOFTWARE**

**Arquitetura Limpia
(Clean Architecture) I-II**

Prof. Helder Prado Santos

La responsabilidad por la idoneidad, originalidad
y licitud de los contenidos didácticos presentados es del profesor.

Prohibida la reproducción, total o parcial, sin autorización.

Ley nº 9610/98

| El Problema del Acoplamiento

Síntomas de una Mala Arquitectura

- **Rigidez:** El sistema es difícil de cambiar porque cada cambio afecta a muchas otras partes.
- **Fragilidad:** Los cambios quiebran partes del sistema que no tienen relación conceptual con la parte alterada.
- **Inmovilidad:** Es difícil reutilizar el sistema en otros proyectos porque implica muchos riesgos y esfuerzo.

Objetivo: Transformar el "Big Ball of Mud" en un sistema estructurado.

Nobody:
My code:



| ¿Qué es la Arquitectura Limpia?

Propuesta por **Robert C. Martin (Uncle Bob)** en 2012, no es una "regla nueva", sino una consolidación de buenas prácticas (Hexagonal, Onion, Screaming Architecture).



Independencia

El sistema no depende de bibliotecas o *frameworks*. Son herramientas, no el núcleo.



Testabilidad

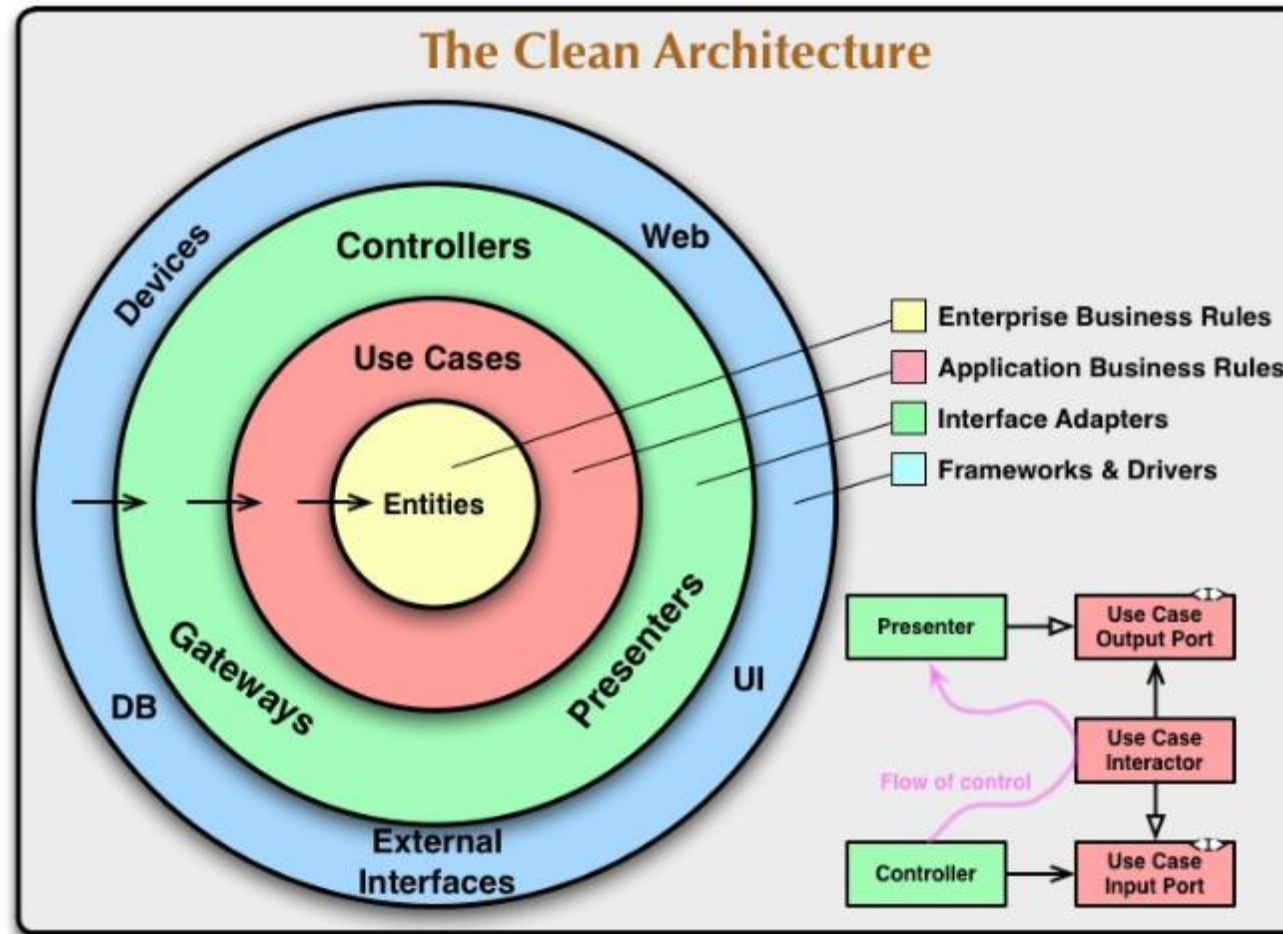
Las reglas de negocio pueden ser testadas sin UI, BD, Servidor Web o cualquier elemento externo.



Independencia de BD

Puedes cambiar Oracle por Mongo, o SQL Server por CouchDB. Las reglas de negocio no saben lo que es una BD.

| El Famoso Diagrama



| La Regla de Oro: Dependencia

"Las dependencias de código fuente deben apuntar solo hacia adentro."

- Nada en un círculo interno puede saber algo sobre un círculo externo.
- Los nombres de clases, funciones o variables declaradas externamente no deben mencionarse internamente.
- El nivel más alto (Reglas) no depende del nivel más bajo (Detalles).

Violación Común:

Importar una biblioteca SQL dentro de su Entidad de Dominio.

| 1. Entidades (Dominio)

Reglas de Negocio Corporativas

Son los objetos de negocio de la aplicación. Encapsulan las reglas más generales y de alto nivel.

- Si es una aplicación corporativa, las entidades pueden ser utilizadas por diferentes aplicaciones de la empresa.
- Si es una aplicación única, son los objetos del dominio principal.
- **No cambian** cuando la página cambia, cuando la seguridad cambia o cuando la base de datos cambia.



Entidades: Código Práctico

```
from uuid import UUID
from domain.task.task_entity import Task
from typing import List

class User:

    id: UUID
    name: str
    tasks: List[Task]

    def __init__(self, id: UUID, name: str):
        self.id = id
        self.name = name
        self.tasks = []
        self.validate()

    def validate(self):
        if not isinstance(self.id, UUID):
            raise Exception("id must be an UUID")

        if not isinstance(self.name, str) or len(self.name) == 0:
            raise Exception("name is required")

    def collect_tasks(self, tasks: List[Task]) -> None:
        self.tasks.extend(tasks)

    def count_pending_tasks(self) -> int:
        count = 0
        for task in self.tasks:
            if task.completed is not True:
                count += 1
```

Observaciones:

- Poseen identidad propia
- Concentran reglas y comportamientos de negocio
- Son independientes de infraestructura
- Deben proteger su estado interno

| 2. Casos de Uso (Aplicación)

Reglas de Negocio de la Aplicación

Contienen reglas específicas de la aplicación. Orquestan el flujo de datos hacia y desde las entidades.

- Dirigen a las entidades para usar sus reglas de negocio corporativas.
- No afectan a las entidades.
- No se ven afectados por UI, BD o *Frameworks*.
- Ejemplos: *Crear Pedido, Matricular Alumno, Generar Extracto*.



Casos de Uso: Inversión de Dependencia

Antes de implementar el caso de uso, definimos la interfaz del repositorio. El Caso de uso **no sabe** cómo guardar en la base de datos, solo sabe que **necesita** guardar.

```
import { User } from '../domain/entities/User';

// Contrato que la capa de Infra debe cumplir
export interface UserRepositoryInterface {
  save(user: User): Promise<void>;
  findByEmail(email: string): Promise<User | null>;
}
```

Esto garantiza que el Caso de Uso dependa de una abstracción, no de una implementación (Principio "D" de SOLID).

Casos de Uso: Implementación

```
from domain.__seedwork.use_case_interface import UseCaseInterface
from domain.user.user_repository_interface import
UserRepositoryInterface
from usecases.user.add_user.add_user_dto import (
    AddUserInputDto,
    AddUserOutputDto
)
from domain.user.user_entity import User
import uuid

class AddUserUseCase(UseCaseInterface):

    def __init__(self, user_repository: UserRepositoryInterface):
        self.user_repository = user_repository

    def execute(self, input: AddUserInputDto) -> AddUserOutputDto:

        user = User(
            id=uuid.uuid4(),
            name=input.name
        )

        self.user_repository.add_user(user=user)

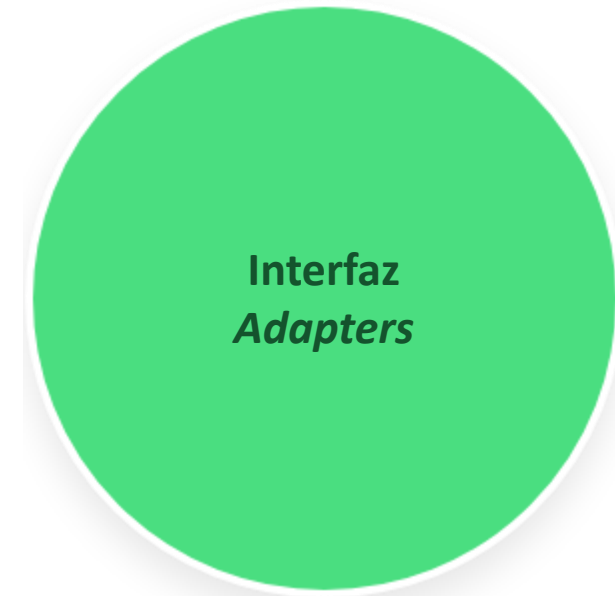
        return AddUserOutputDto(
            id=user.id,
            name=user.name
```

| 3. Adaptadores de Interfaz

Conversores de Datos

El objetivo es convertir datos del formato más conveniente para los casos de uso y entidades, al formato más conveniente para agentes externos (Web, BD).

- **Controllers:** Reciben el *input* (HTTP *Request*) y lo convierten a DTO del Caso de Uso.
- **Presenters:** Reciben el retorno del Caso de Uso y lo formatean para la *View* (HTML, JSON).
- **Gateways:** Implementaciones de interfaces para servicios externos.



| Adaptadores: *Web Controller*

```
export class UserController {
  constructor(private useCase: RegisterUser) {}

  // Recibe objetos de framework (Request, Response)
  async handle(request: Request, response: Response) {
    try {
      const { name, email } = request.body;

      // Llama al caso de uso (Agnóstico a HTTP)
      const result = await this.useCase.execute({
        id: UUID.gen(),
        name,
        email,
      });

      return response.status(201).json(result);
    } catch (e) {
      return response.status(400).json({
        error: e.message,
      });
    }
  }
}
```

Responsabilidades:

- Desacoplar el HTTP del Dominio.
- Extraer datos de la solicitud.
- Invocar el Caso de Uso.
- Devolver la respuesta correcta.

| 4. *Frameworks y Drivers* (Infra)

Donde viven los detalles

La capa más externa. Generalmente compuesta por *frameworks* y herramientas como:

- Base de Datos (Postgres, Mongo).
- *Web Frameworks* (Express, NestJS, Spring).
- Dispositivos (UI, Impresoras).

No escribimos mucho código aquí, aparte del código de "pegar" que se comunica con los círculos internos.



Infra: Implementación de Repositorio

Aquí implementamos la interfaz definida en la capa de Aplicación.

```
import { UserRepositoryInterface } from '../..application/repositories';
import { User } from '../..domain/entities/User';
import { UserModel } from '../mongoose/UserSchema';

// Implementación concreta usando MongoDB (lib externa)
export class MongoUserRepository implements UserRepositoryInterface {
  async save(user: User): Promise<void> {
    // Convierte Entidad de Dominio → Modelo de Mongo
    await UserModel.create({
      _id: user.id,
      name: user.name,
      email: user.email,
    });
  }

  async findByEmail(email: string): Promise<User | null> {
    const data = await UserModel.findOne({ email });

    if (!data) {
      return null;
    }

    // Reconstruye la Entidad de Dominio
    return new User(
      data._id.toString(),
      data.name,
      data.email
    );
  }
}
```

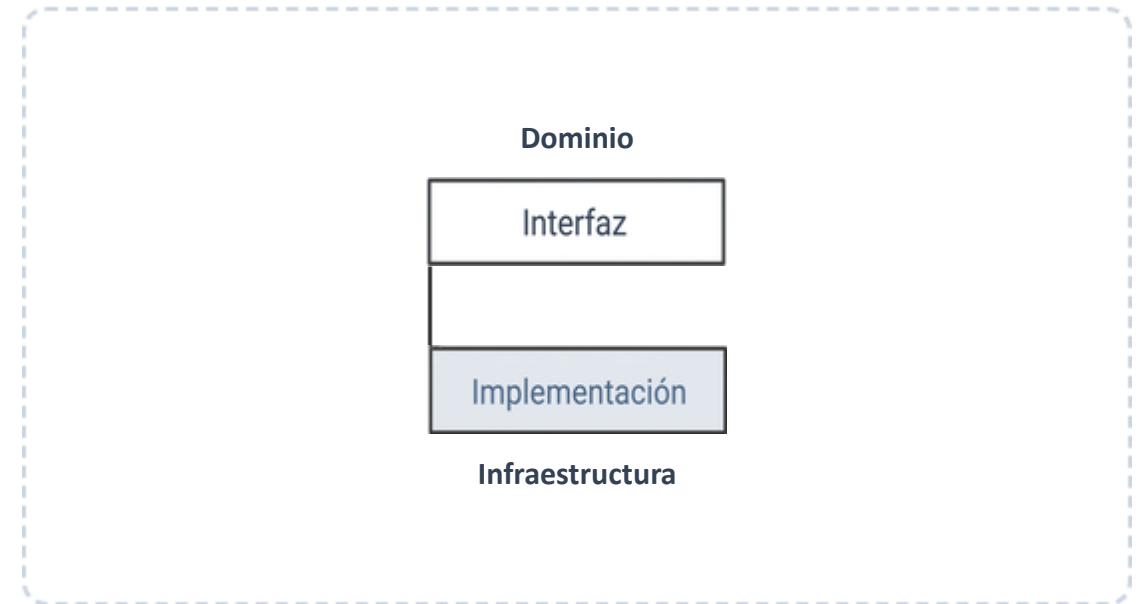
| Cruzando Límites (*Boundaries*)

Flujo de Control vs. Dependencia

El flujo de control (*runtime*) puede ir en cualquier dirección (*Controller* llama a *UseCase*, *UseCase* llama a Repositorio).

Sin embargo, la **dependencia de código fuente** siempre debe apuntar en contra del flujo de control cuando cruzamos límites hacia afuera.

¿Cómo? Polimorfismo (Interfaces). El *Use Case* llama a una interfaz *RepositoryInterface* (que está en la capa *Domain*), pero la implementación *MongoRepository* está en la capa Infra.



| Estrategia de Pruebas



Pruebas Unitarias

Prueban Entidades y Casos de Uso. Son rápidas (ms). No usan *Mocks* complejos, solo datos puros.



Pruebas de Integración

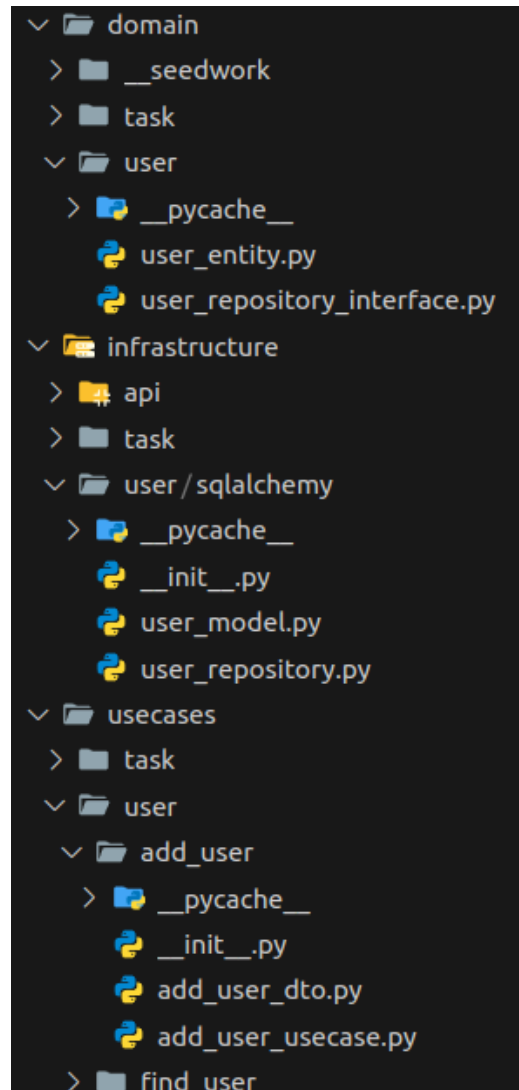
Prueban la comunicación entre Adaptadores y Casos de Uso. Verifican si el *Controller* pasa el DTO correcto.



Pruebas E2E

Prueban la Infraestructura. Levantan la base de datos real y realizan solicitudes HTTP reales.

Estructura de Carpetas Sugerida



Screaming Architecture

La estructura de carpetas debe "gritar" sobre lo que hace el sistema, no qué *framework* utiliza.

Al abrir el proyecto, debe ver "Usuarios", "Pedidos", "Productos", y no solo "*Models*", "*Views*", "*Controllers*".

| *Trade-offs* (No todo es color de rosa)

Ventajas

- Desacoplamiento total.
- Testabilidad extrema.
- Fácil mantenimiento a largo plazo.
- Independencia de *Frameworks*.

Desventajas

- Curva de aprendizaje empinada.
- Mucho "*boilerplate*" (Muchos archivos).
- *Overhead* mental para *CRUDs* simples.
- Duplicación de código (*DTO* vs *Entity*).

| ¿Cuándo usar Arquitectura Limpia?

ÚSELA CUANDO:

- Proyectos medianos o grandes, con reglas de negocio complejas.
- Sistemas con larga vida útil, que necesitarán evolucionar con el tiempo.
- Equipos con varios desarrolladores, facilitando la organización y el mantenimiento.
- Necesidad de pruebas automatizadas, especialmente pruebas de negocio.

EVÍTELA CUANDO:

- Proyectos pequeños o *MVPs*, donde la simplicidad y la velocidad son prioridad.
- Aplicaciones muy simples (*CRUD* básico) sin reglas complejas.
- Plazos muy cortos, donde la sobrecarga arquitectónica dificulta.
- Equipos sin madurez técnica, pudiendo generar confusión y *overengineering*.

| Recapitulación

- **Entidades:** El núcleo del dominio, conteniendo las reglas de negocio más estables.
- **Casos de Uso:** Orquestan las reglas de negocio y definen el flujo de la aplicación.
- **Adaptadores:** Convierten datos entre el mundo externo y el dominio interno.
- **Infraestructura:** Implementaciones técnicas y herramientas reemplazables.
- **Reglas de Dependencia:** Las dependencias siempre apuntan al núcleo de la aplicación.

"Una buena arquitectura permite posponer decisiones críticas."



Sugerecias de Lecturas/Referencias

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2018.

MARTIN, Robert C. Código limpo: habilidades práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

EVANS, Eric. Domain-Driven Design: atacando as complexidades no coração do software. Rio de Janeiro: Alta Books, 2016.

MARTIN, Robert C. (Uncle Bob). The Clean Code Blog. Disponível em: <https://blog.cleancoder.com>

¡Gracias!

Helder Prado Santos | [linkedin.com/in/helderprado](https://www.linkedin.com/in/helderprado)

MBAUSP
ESALQ